Introduction to Deep Learning

Assignment 1: Building Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) with PyTorch

October 2026

Introduction

In this assignment, you are going to use the **PyTorch** deep learning library (`torch.nn`, `torch.optim`, `torch.utils.data`, `torchvision`) to build various neural network models for classification and regression tasks on image datasets.

You are advised to use the following references for practical guidance and theoretical foundations:

- Simon J.D. Prince: *Understanding Deep Learning* (MIT Press) — specifically Chapters 3–5 (MLPs), Chapter 10 (CNNs), and Chapter 6–7 (Optimization and Loss Functions).
- Official PyTorch Tutorials: https://pytorch.org/tutorials/beginner/basics/intro.html (covering Tensors, Datasets & DataLoaders, Transforms, Model Building, Autograd, and Optimization).

The key objectives of this assignment:

- Learn how to define, train, and evaluate neural networks in PyTorch using `nn.Module` and standard training loops.
- Gain practical experience by comparing various MLP and CNN architectures, gaining intuition for manually tuning hyperparameters (learning rates, optimizers, regularization) and their effects on model performance.
- Apply this knowledge to develop a more complex CNN for the "tell-the-time" problem.

Note: This assignment involves computationally intensive model training; therefore, you are recommended to find a suitable platform with GPU acceleration:

1. **Lab Computers:** The lab machines have a shared environment with all necessary packages and GPU acceleration pre-installed. See Brightspace → General information → Instructions for remote connection details.
2. **Google Colab / DeepNote:** Select GPU runtime under `Runtime` → `Change runtime type` → `GPU`.
3. **Local Machine:** You can use your own computer with an NVIDIA GPU or CPU. In your PyTorch code, manage hardware devices seamlessly via:

   ```
   device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
   ```

   and ensure all models and data tensors are transferred using `.to(device)`.

Task 1: Learn the basics of PyTorch

Start by studying Chapters 3–5 and Chapter 10 of Prince's *Understanding Deep Learning*, along with the official PyTorch tutorials. Explore the core modules provided by PyTorch: `torch.nn` (layers, loss functions, containers), `torch.optim` (SGD, Adam, AdamW, learning rate schedulers), and `torchvision` (datasets and vision transforms).

1. Familiarize yourself with standard PyTorch workflow examples for image classification on the MNIST dataset (defining an `nn.Module`, writing a training epoch with optimizer zero-grad, loss backward, optimizer step, and evaluating on a test set).

2. Next, work with two benchmark datasets loaded via `torchvision.datasets`: **Fashion-MNIST** and **CIFAR-10**.

   - Fashion-MNIST contains 28×28 grayscale images across 10 categories (60,000 train / 10,000 test).
   - CIFAR-10 contains $32 \times 32 \times 3$ RGB images across 10 categories (50,000 train / 10,000 test).

   Apply two reference network architectures to the Fashion-MNIST dataset:

   (a) A multi-layer perceptron (MLP) built with `nn.Sequential` or subclassing `nn.Module` with linear layers (`nn.Linear`) and non-linear activations.

   (b) A convolutional neural network (CNN) incorporating 2D convolutions (`nn.Conv2d`), pooling layers (`nn.MaxPool2d`), and linear classifier heads.

   Experiment systematically with both networks by exploring different choices for:

   - Parameter initializations (`torch.nn.init`)
   - Activation functions (`nn.ReLU`, `nn.LeakyReLU`, `nn.GELU`, etc.)
   - Optimizers and learning rates (`optim.SGD` with momentum, `optim.Adam`, `optim.AdamW`)
   - Regularization techniques (`nn.Dropout`, weight decay / L_2 penalty)
   - Architectural variations (depth, width, number and size of convolutional kernels)

   Split the training data to create a validation set (10% of the training data) to monitor overfitting and guide hyperparameter choices. After identifying your best-performing hyperparameter configurations, select the 3 best ones and train models on the CIFAR-10 dataset to examine whether the performance gains transfer across datasets.

   Discuss your findings in the report (e.g., why do certain architectures or hyperparameters generalize differently on CIFAR-10 compared to Fashion-MNIST?).

   Note: The goal is **not** merely to achieve the highest possible test accuracy, but to gain solid practical intuition for tuning deep networks, conducting systematic experiments, and documenting findings clearly.

Task 2: Develop a "Tell-the-Time" Network

We have produced a collection of images of an analog clock along with labels corresponding to the time displayed. Your task is to design, train, and evaluate a PyTorch CNN that predicts the time shown as accurately as possible.

The dataset can be found here and consists of 18,000 grayscale images (`images.npy`, sized $18000 \times 150 \times 150$ or downsampled $18000 \times 75 \times 75$). The labels (`labels.npy`, sized 18000×2)

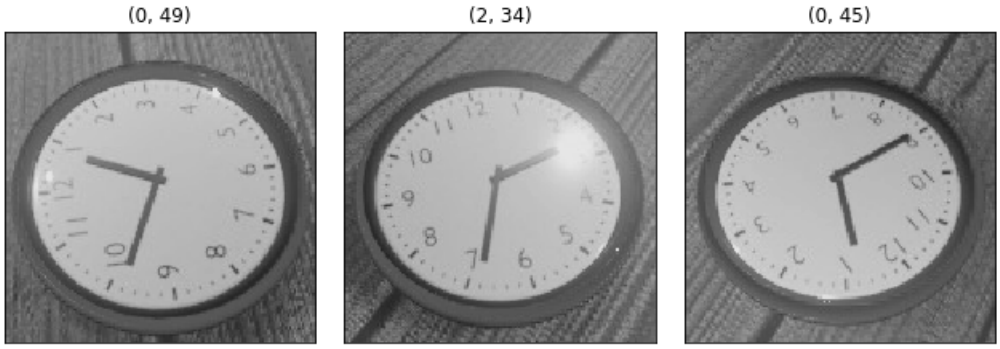


Figure 1: Random sample images from the clock collection with the corresponding time (hours, minutes) displayed.

contain the integer hour and minute displayed. Each image is rendered with varying perspective angles, rotations, and lighting reflections.

Split your data into **80% training, 10% validation, and 10% test** sets. Remember to shuffle the dataset, as the raw files may be ordered. We suggest prototyping on the 75×75 images and reporting final results on the 150×150 dataset.

Subtasks and Model Formulations

The problem of predicting the time can be formulated in several ways:

1. **Classification:** Treat time prediction as an n-class classification problem. Start with coarse grouping (e.g., 30-minute bins resulting in 24 classes, using `nn.CrossEntropyLoss`), and then increase resolution towards minute-level classification (up to 720 classes). What limitations does a direct high-cardinality classification representation exhibit?

2. **Regression:** Formulate time as continuous scalar values (e.g., mapping `"03:00"` $\rightarrow 3.0$ and `"05:30"` $\rightarrow 5.5$). What loss functions (`nn.MSELoss`, `nn.L1Loss`, `nn.SmoothL1Loss`) are appropriate, and what boundary discontinuity issues arise around 12:00 / 0:00?

3. **Multi-Head Models in PyTorch:** Construct an `nn.Module` featuring a shared convolutional feature extractor with two distinct output heads: one for predicting hours and another for predicting minutes. In PyTorch, define the shared backbone in `__init__` and return both head outputs in `forward()`:

   ```
   out_hour, out_minute = model(x)
   ```

   Combine the individual losses into a joint objective: $\mathcal{L} = \mathcal{L}_{\text{hour}} + \alpha \cdot \mathcal{L}_{\text{minute}}$. Compare multi-head performance against single-output regression and classification models.

4. **Label Transformation via Trigonometric Encoding (Optional):** Represent the cyclic nature of clock hands by mapping hours and minutes to angles on the unit circle:

   $$\theta_h = \frac{2\pi \cdot (\text{hour} + \text{minute}/60)}{12}, \quad \theta_m = \frac{2\pi \cdot \text{minute}}{60}$$

   The network can predict $(\sin\theta_h, \cos\theta_h, \sin\theta_m, \cos\theta_m)$. How should the output layer and activations be designed for this representation?

Evaluation Metric: Common Sense Accuracy

Regardless of which model architecture or training loss you use, evaluate and report your models using the **"common sense" error**: the absolute time difference between predicted time and true time taking into account modulo 12-hour wrap-around (e.g., the difference between predicted 11:55 and target 0:05 is 10 minutes, not 11 hours and 50 minutes). Minimizing this error is the core objective of this assignment.

Deliverables

Your submission should consist of:

1. A concise written report in PDF format (at most 10 pages).
2. Neatly organized and reproducible Jupyter notebook(s) with PyTorch code.
3. Exported `.py` script files for each notebook.

Do not use archive compression (`.zip`/`.rar`); submit each file individually on Brightspace.